

ΜΥΕ023 – ΠΑΡΑΛΛΗΛΑ ΣΥΣΤΗΜΑΤΑ ΚΑΙ
ΠΡΟΓΡΑΜΜΑΤΙΣΜΟΣ 2023-24

ΜΡΙ ΚΑΙ ΥΒΡΙΔΙΚΟΣ
ΠΡΟΓΡΑΜΜΑΤΙΣΜΟΣ
ΣΕΤ ΑΣΚΗΣΕΩΝ #3

Περιεχόμενα

Εισαγωγή	2
Άσκηση 1: Θόλωση εικόνων με MPI	3
Το πρόβλημα	3
Ανάλυση Κώδικα	3
Γραφικές Παραστάσεις – Χρονομετρήσεις	9
Συμπεράσματα.....	11
Άσκηση 2: Υβριδικός πολλαπλασιασμός πινάκων MPI/OpenMP.....	12
Το πρόβλημα	12
Ανάλυση Κώδικα	13
Γραφικές Παραστάσεις - Χρονομετρήσεις.....	15
Συμπεράσματα.....	17
Άσκηση 3: Μονόπλευρες επικοινωνίες.....	18
Τι είναι οι Μονόπλευρες επικοινωνίες.....	18
Λειτουργίες του MPI για Μονόπλευρες επικοινωνίες	18
Χρήση Μονόπλευρων Επικοινωνιών σε Εφαρμογές	19
Πηγές	19

Εισαγωγή

Η 3^η σειρά ασκήσεων εστιάζει στον παράλληλο προγραμματισμό με χρήση του MPI και υβριδικών τεχνικών, και αποτελείται από τρία μέρη. Κάθε μέρος εξετάζει διαφορετικές πτυχές αυτής της τεχνολογίας για την ανάπτυξη αποδοτικών παράλληλων προγραμμάτων.

Το πρώτο μέρος επικεντρώνεται στην παραλληλοποίηση της διαδικασίας θόλωσης εικόνας χρησιμοποιώντας το MPI. Ο στόχος αυτής της άσκησης είναι να τροποποιηθεί το σειριακό πρόγραμμα Gaussian blur ώστε να εκτελείται παράλληλα μέσω του MPI, αξιοποιώντας τις δυνατότητες πολλαπλών διεργασιών για την επιτάχυνση της επεξεργασίας

Το δεύτερο μέρος ασχολείται με τον υβριδικό προγραμματισμό, συνδυάζοντας το MPI και το OpenMP για τον πολλαπλασιασμό πινάκων. Θα υλοποιηθεί ένα πρόγραμμα πολλαπλασιασμού πίνακα επί πίνακα, εκμεταλλευόμενοι τόσο την κατανομημένη υπολογιστική ισχύ μέσω MPI όσο και την πολυπύρηνη επεξεργασία μέσω OpenMP.

Το τρίτο μέρος εστιάζει στις μονόπλευρες επικοινωνίες (one-sided communication) και την απομακρυσμένη προσπέλαση μνήμης (Remote Memory Access - RMA) στο MPI. Θα ερευνηθούν και θα κατανοηθούν οι μονόπλευρες επικοινωνίες, κατά τις οποίες μία διεργασία μπορεί να προσπελάσει άμεσα τη μνήμη μιας άλλης διεργασίας χωρίς την ανάγκη άμεσης συμμετοχής της τελευταίας κατά τη στιγμή της πρόσβασης.

Για τις μετρήσεις χρησιμοποιήθηκαν τα παρακάτω σύστημα:

Όνομα υπολογιστή	opti3060ws01 έως και opti3060ws08
Cpu	Intel(R) Core(TM) i3-8300 CPU @ 3.70GHz
Cores	4
Μεταφραστής	gcc 11.2.0

Άσκηση 1: Θόλωση εικόνων με MPI

Το πρόβλημα

Το πρόβλημα που τίθεται στην άσκηση 1 αφορά την παράλληλη εφαρμογή του Gaussian blur για την επεξεργασία εικόνων με χρήση του MPI.

Στόχος είναι να παραλληλοποιηθεί η διαδικασία θόλωσης της εικόνας, επιτρέποντας σε πολλαπλές διεργασίες να επεξεργαστούν τμήματα της εικόνας ταυτόχρονα, επιταχύνοντας έτσι την μείωση του συνολικού χρόνου επεξεργασίας

Η παράλληλη συνάρτηση `gaussian_blur_mpi()` θα χρησιμοποιεί τις δυνατότητες του MPI για να διαμοιράσει την επεξεργασία σε πολλαπλούς κόμβους.

Η παραλληλοποίηση θα εφαρμοστεί στο εξωτερικό for loop, με κάθε διεργασία να αναλαμβάνει ένα συνεχόμενο μπλοκ γραμμών της εικόνας.

Ανάλυση Κώδικα

1. Κώδικας `main()` :

- Αρχικοποίηση του MPI:

```
/* MPI Initialization */  
int myid, nproc;  
MPI_Init(&argc, &argv);  
MPI_Comm_rank(MPI_COMM_WORLD, &myid);  
MPI_Comm_size(MPI_COMM_WORLD, &nproc);
```

- **Αρχικοποίηση δεδομένων εισόδου και προαπαιτήμενων ενεργειών(διεργασία 0):**

Η διεργασία 0 (η κύρια διεργασία) εκτελεί όλες τις προκαταρκτικές ενέργειες για την προετοιμασία της εικόνας και εκτελεί τη σειριακή μέθοδο θόλωσης.

Συγκεκριμένα, διαβάζει την είσοδο, ελέγχει την εγκυρότητα των παραμέτρων, προετοιμάζει τις δομές δεδομένων, εκτελεί τη σειριακή θόλωση της εικόνας και αποθηκεύει τα αποτελέσματα.

```
if(myid == 0) {
    if (argc < 3) {...}

    inputfile = argv[2];

    if (radius < 0) {
        fprintf(stderr, "Radius should be an integer >= 0; exiting.");
        MPI_Finalize();
        exit(1);
    }

    noextfname = remove_ext( str: inputfile, extsep: '.', pathsep: '/');
    sprintf(seqoutfile, "%s-r%d-serial.bmp", noextfname, radius);
    sprintf(paroutfile_mpi, "%s-r%d-mpi.bmp", noextfname, radius);

    bmp_read_img_from_file(inputfile, img: &imgin);
    bmp_clone_empty_img( imgin: &imgin, imgout: &imgout);
    bmp_clone_empty_img( imgin: &imgin, imgout: &pimgout_mpi);
    bmp_rgb_alloc( img: &imgin);
    bmp_rgb_alloc( img: &imgout);
    bmp_rgb_alloc( img: &pimgout_mpi);

    printf("<<< Gaussian Blur (h=%d,w=%d,r=%d) >>>\n", imgin.header.height,
        imgin.header.width, radius);

    /* Image data to R,G,B */
    bmp_rgb_from_data( img: &imgin);

    /* Run & time serial Gaussian Blur */
    exectime_serial = timeit( func: gaussian_blur_serial, radius, imgin: &imgin, imgout: &imgout);
    printf("Total execution time (sequential): %lf\n", exectime_serial);
    /* Save the results (serial) */
    bmp_data_from_rgb( img: &imgout);
    bmp_write_data_to_file( fname: seqoutfile, img: &imgout);
}
```

- Παράλληλη εκτέλεση και χρονομέτρηση της gaussian blur:

```
double start = MPI_Wtime();
gaussian_blur_mpi(radius, imgin: &imgin, imgout: &pimgout_mpi, myid, nproc);
double end = MPI_Wtime();
exectime_mpi = end - start;
```

- Εκτύπωση χρονομετρήσεων και τερματισμός MPI(διεργασία 0):

```
if(myid == 0) {
    bmp_data_from_rgb( img: &pimgout_mpi);
    bmp_write_data_to_file( fname: paroutfile_mpi, img: &pimgout_mpi);

    printf("Total execution time (sequential): %lf\n", exectime_serial);
    printf("Total execution time (mpi): %lf\n", exectime_mpi);

    bmp_img_free( img: &imgin);
    bmp_img_free( img: &imgout);
    bmp_img_free( img: &pimgout_mpi);
}

MPI_Finalize();
```

2. Κώδικας gaussian_blur_mpi:

- Προσδιορισμός διαστάσεων εικόνας από τη διεργασία 0:

Η διεργασία 0 είναι υπεύθυνη για τη φόρτωση της εικόνας, οπότε γνωρίζει τις διαστάσεις της και πρέπει να τις μεταδώσει στις άλλες διεργασίες.

```
if(myid == 0){
    width = imgin->header.width;
    height = imgin->header.height;
}
```

- **Broadcast των διαστάσεων εικόνας από τη διεργασία 0 σε όλες τις άλλες:**

```
MPI_Bcast(&width, 1, MPI_INT, 0, MPI_COMM_WORLD);
MPI_Bcast(&height, 1, MPI_INT, 0, MPI_COMM_WORLD);
```

- **Υπολογισμός των γραμμών που θα επεξεργαστεί κάθε διεργασία:**

```
start_row = myid * (height / nproc);
end_row = (myid + 1) * (height / nproc);
if (myid == nproc - 1) {
    end_row = height;
}
rows_per_proc = end_row - start_row;
```

- **Δέσμευση μνήμης για διεργασίες εκτός της 0:**

Οι διεργασίες που δεν είναι η 0 χρειάζονται τοπική μνήμη για να αποθηκεύσουν τα δεδομένα της εικόνας και τα αποτελέσματα του υπολογισμού.

```
if(myid !=0) {
    imgin->red = (unsigned char *) calloc(height * width, sizeof(unsigned char));
    imgin->green = (unsigned char *) calloc(height * width, sizeof(unsigned char));
    imgin->blue = (unsigned char *) calloc(height * width, sizeof(unsigned char));

    imgout->red = (unsigned char *) calloc(height * width, sizeof(unsigned char));
    imgout->green = (unsigned char *) calloc(height * width, sizeof(unsigned char));
    imgout->blue = (unsigned char *) calloc(height * width, sizeof(unsigned char));
}
```

- **Broadcast των δεδομένων της εικόνας από τη διεργασία 0 σε όλες τις διεργασίες:**

```
MPI_Bcast(imgin->red, width * height, MPI_BYTE, 0, MPI_COMM_WORLD);
MPI_Bcast(imgin->green, width * height, MPI_BYTE, 0, MPI_COMM_WORLD);
MPI_Bcast(imgin->blue, width * height, MPI_BYTE, 0, MPI_COMM_WORLD);
```

- **Υπολογισμός της Gaussian Blur για το τμήμα της εικόνας που αναλογεί σε κάθε διεργασία:**

```
for (i = start_row; i < end_row; i++)
{
```

- **Συγκέντρωση των αποτελεσμάτων από όλες τις διεργασίες στην διεργασία(0):**

Το πλήθος των γραμμών άρα και των στοιχείων που είχε κάθε διεργασία να επεξεργαστεί και να αποθηκεύσει τοπικά, δεν είναι ίδιο για κάθε μία, άρα για αυτόν τον λόγο χρησιμοποιήθηκε Gatherν έναντι της απλής Gather.

Για την χρήση της Gatherν χρησιμοποιούμε τους πίνακες sendcounts και displs. Ο sendcounts καθορίζει πόσα στοιχεία θα στείλει η κάθε διεργασία. Ο displs καθορίζει το σημείο που θα τοποθετηθούν τα δεδομένα της η κάθε διεργασία στην τελική εικόνα.

Με αυτόν τον τρόπο, εξασφαλίζεται ότι όλα τα τμήματα της εικόνας θα συνδυαστούν σωστά στην τελική έξοδο.

```
int *sendcounts = (int *) malloc(nproc * sizeof(int));
int *displs = (int *) malloc(nproc * sizeof(int));
int offset = 0;
for (i = 0; i < nproc; i++) {
    if (i == nproc - 1) {
        sendcounts[i] = (height - i * (height / nproc)) * width;
    } else {
        sendcounts[i] = (height / nproc) * width;
    }
    displs[i] = offset;
    offset += sendcounts[i];
}
```



```

MPI_Gatherv(imgout->red + start_row * width, rows_per_proc * width, MPI_BYTE,
            imgout->red, sendcounts, displs, MPI_BYTE, 0, MPI_COMM_WORLD);
MPI_Gatherv(imgout->green + start_row * width, rows_per_proc * width, MPI_BYTE,
            imgout->green, sendcounts, displs, MPI_BYTE, 0, MPI_COMM_WORLD);
MPI_Gatherv(imgout->blue + start_row * width, rows_per_proc * width, MPI_BYTE,
            imgout->blue, sendcounts, displs, MPI_BYTE, 0, MPI_COMM_WORLD);

```

- **Αποδέσμευση μνήμης για τις εικόνες στις διεργασίες εκτός της 0:**

```

if(myid != 0){
    free(imgin->red);
    free(imgin->green);
    free(imgin->blue);
    free(imgout->red);
    free(imgout->green);
    free(imgout->blue);
}

```

- **Εκτύπωση communication time και computation time:**

```

if(myid == 0){
    printf("Communications time = %f \n", comm_time);
    printf("Computations time = %f \n", comp_time);
}

```

Σημειώσεις :

Περαιτέρω σχόλια για τις εντολές που χρησιμοποιήσαμε έχουν προστεθεί πάνω στον παραδοτέο κώδικα .

Μια ακόμα πιο βέλτιστη προσέγγιση θα ήταν να χρησιμοποιήσουμε Scatterv έναντι του broadcast στην αρχική αποστολή των δεδομένων από την διεργασία 0 προς όλες τις άλλες (το μέγεθος των δεδομένων αποστολής καθώς και ο χώρος που θα καταλάμβαναν τοπικά τα δεδομένα των εικόνων στις διεργασίες θα ήταν μικρότερο).

Γραφικές Παραστάσεις – Χρονομετρήσεις

- Σειριακή εκτέλεση:

Total Execution time = 15,2525 seconds

- 2 επί 4 = 8 διεργασίες:

	1 ^η εκτέλεση	2 ^η εκτέλεση	3 ^η εκτέλεση	4 ^η εκτέλεση	M.O
Communications time	0,670436	0,330042	0,434518	0,379928	0,453
Computations time	1,891907	1,891752	1,903881	1,897945	1,896
Total Execution time	2,562357	2,221809	2,338415	2,277887	2,35

Ποσοστό Communications Time	19,31%
Ποσοστό Computations Time	80,69%

- 4 επί 4 = 16 διεργασίες:

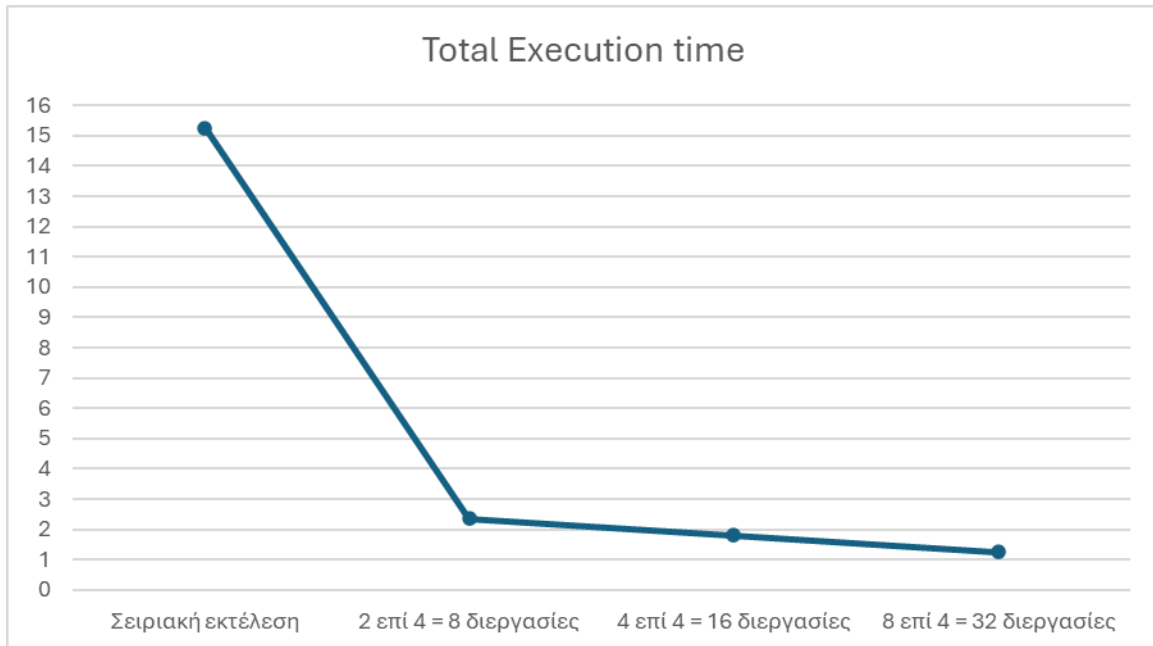
	1 ^η εκτέλεση	2 ^η εκτέλεση	3 ^η εκτέλεση	4 ^η εκτέλεση	M.O
Communications time	0,862129	0,684774	0,914420	1,009977	0,867
Computations time	0,950835	0,940654	0,942025	0,944898	0,944
Total Execution time	1,812980	1,625443	1,856459	1,954891	1,81

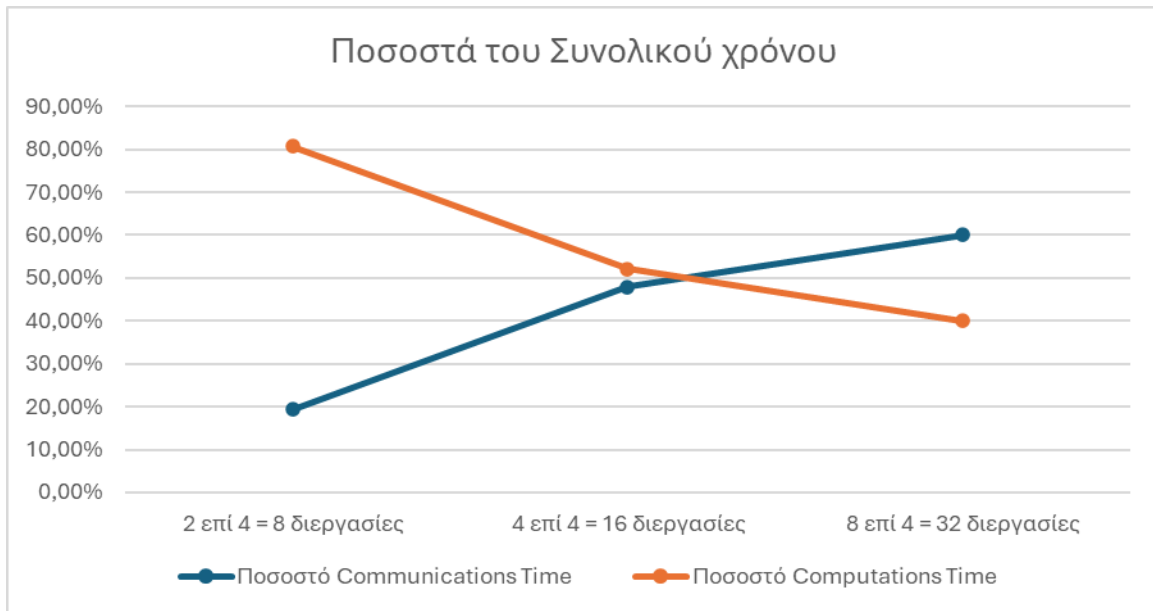
Ποσοστό Communications Time	47,88%
Ποσοστό Computations Time	52,12%

- 8 επί 4 = 32 διεργασίες:

	1 ^η εκτέλεση	2 ^η εκτέλεση	3 ^η εκτέλεση	4 ^η εκτέλεση	M.O
Communications time	0,784080	0,813364	0,670312	0,744786	0,753
Computations time	0,494949	0,502191	0,493876	0,514816	0,501
Total Execution time	1,279052	1,315569	1,164202	1,259619	1,25

Ποσοστό Communications Time	60,03%
Ποσοστό Computations Time	39,97%





Συμπεράσματα

1. Αύξηση της Απόδοσης με Παράλληλη Εκτέλεση:

Όπως φαίνεται από τον συνολικό χρόνο εκτέλεσης (total execution time), η χρήση περισσότερων διεργασιών μειώνει τον χρόνο εκτέλεσης σε σύγκριση με τη σειριακή εκτέλεση. Αυτό αποδεικνύει ότι η παράλληλη επεξεργασία βελτιώνει την απόδοση της θόλωσης εικόνων.

2. Αύξηση του Χρόνου Επικοινωνίας:

Παρατηρούμε ότι με την αύξηση του αριθμού των διεργασιών, ο χρόνος επικοινωνίας αυξάνεται. Για παράδειγμα, με 32 διεργασίες, ο χρόνος επικοινωνίας είναι 60.03% του συνολικού χρόνου, σε σύγκριση με 19.31% για 8 διεργασίες. Αυτό συμβαίνει επειδή η κατανομή των δεδομένων (broadcast) και η συλλογή των αποτελεσμάτων (gather) απαιτούν επικοινωνία μεταξύ περισσότερων διεργασιών.

3. Μείωση του Χρόνου Υπολογισμών:

Ο χρόνος υπολογισμών μειώνεται με την αύξηση των διεργασιών, καθώς το φορτίο εργασίας κατανέμεται σε περισσότερες διεργασίες. Αυτό επιτρέπει τη γρηγορότερη επεξεργασία των δεδομένων.

Συνολικά, η παράλληλη επεξεργασία με MPI βελτιώνει σημαντικά την απόδοση της θόλωσης εικόνων, μειώνοντας τον συνολικό χρόνο εκτέλεσης. Ωστόσο, καθώς αυξάνεται ο αριθμός των διεργασιών, ο χρόνος επικοινωνίας αυξάνεται, περιορίζοντας τα οφέλη της παράλληλης επεξεργασίας.

Άσκηση 2: Υβριδικός πολλαπλασιασμός πινάκων MPI/OpenMP

Το πρόβλημα

Το πρόβλημα που τίθεται στην άσκηση 2 είναι ο πολλαπλασιασμός δύο πινάκων με χρήση υβριδικού προγραμματισμού MPI/OpenMP.

Ο υβριδικός προγραμματισμός συνδυάζει το MPI για τη διαχείριση διεργασιών σε διαφορετικούς κόμβους και το OpenMP για την παραλληλοποίηση εντός κάθε κόμβου, χρησιμοποιώντας τους πολλαπλούς πυρήνες κάθε κόμβου για επιτάχυνση του υπολογισμού.

Συγκεκριμένα, η αρχιτεκτονική του συστήματος αποτελείται από K κόμβους με N πυρήνες ο καθένας, θα δημιουργηθούν K διεργασίες MPI. Κάθε μία από αυτές θα εκτελεί τον πολλαπλασιασμό πινάκων με τη χρήση του OpenMP, όπου η παράλληλη περιοχή θα εκτελείται από N νήματα.

Ανάλυση Κώδικα

Παρακάτω θα αναφέρουμε τις προσθήκες και την λογική πίσω από αυτές για την μετάβαση από τον κώδικα για τον σειριακό πολλαπλασιασμό πινάκων , στην τελική υλοποίηση της εφαρμογής με χρήση υβριδικού MPI/OpenMP.

1. Προσθήκη MPI και OpenMP Βιβλιοθηκών:

```
#include <mpi.h>
#include <omp.h>
```

2. Δήλωση Μεταβλητών MPI:

```
int myid, nproc;
```

3. Αρχικοποίηση MPI:

```
// We initialize MPI
MPI_Init(&argc, &argv);
MPI_Comm_rank(MPI_COMM_WORLD, &myid); // We get the id of the current process
MPI_Comm_size(MPI_COMM_WORLD, &nproc); // We get the total number of processes
```

4. Διαχείριση Πινάκων εισόδου και Συλλογικών επικοινωνιών.

Η διεργασία με id=0 είναι αυτή διαβάζει και κατέχει τα δεδομένα εισόδου από τα αρχεία των πινάκων.

```
// Process with id 0 reads the matrices from the files
if (myid == 0) {
    if (readmat( fname: Afile, mat: (int *)A, n: N) < 0)
        exit(1 + printf("file problem\n"));
    if (readmat( fname: Bfile, mat: (int *)B, n: N) < 0)
        exit(1 + printf("file problem\n"));
}
```

Έπειτα η διεργασία με `id=0` διαμοιράζει τα δεδομένα στις υπόλοιπες διεργασίες, πιο συγκεκριμένα κάνει broadcast ολόκληρο των πίνακα B και μετέπειτα κάνει scatter τα block δεδομένων από τον πίνακα A που αναλογούν σε κάθε διεργασία.

```
// Process 0 broadcast matrix B to all processes
MPI_Bcast(B, N*N, MPI_INT, 0, MPI_COMM_WORLD);
```

```
// Process 0 scatter the parts of matrix A to all processes
MPI_Scatter(A, work*N, MPI_INT, &A[myid*work][0], work*N, MPI_INT, 0, MPI_COMM_WORLD);
```

5. Παραλληλισμός με OpenMP:

Κάθε διεργασία παραλληλοποιεί το μέρος του υπολογισμού που της αναλογεί με την χρήση της OpenMP. (`num_threads(4)` διότι πετυχαίνουμε την μέγιστη απόδοση καθώς οι υπολογιστές που εκτελούν το πρόγραμμα έχουν 4 cores ο καθένας).

```
// Parallel execution with OpenMP
#pragma omp parallel for private(i, j, k, sum) shared(A, B, C) num_threads(4)
for (i = myid*work; i < (myid+1)*work; i++) {
```

6. Διαχείριση αποτελεσμάτων:

Μετά το πέρας των υπολογισμών κάθε διεργασίας κάνουμε gather τα αποτελέσματα στην διεργασία με `id=0`.

```
// We gather the parts of matrix C from all processes to the process with id 0
MPI_Gather(&C[myid*work][0], work*N, MPI_INT, C, work*N, MPI_INT, 0, MPI_COMM_WORLD);
```

7. Γράψιμο των αποτελεσμάτων σε αρχείο και τερματισμός MPI:

Τα αποτελέσματα από τον πολλαπλασιασμό πινάκων έχουν συγκεντρωθεί στον πίνακα C της διεργασίας 0.

Η οποία είναι αυτή που θα τα γράψει στο αρχείο εξόδου.

```
// Process 0 writes matrix C to the file
if (myid == 0) {
    writemat( fname: Cfile, mat: (int *)C, n: N);
}

MPI_Finalize();
```

8. Κώδικας Χρονομέτρησης:

Για την χρονομέτρηση των επικοινωνιών , των υπολογισμών αλλά και του συνολικού χρόνου του gaussian_blur με MPI χρησιμοποιήθηκε η συνάρτηση MPI_Wtime() .

Γραφικές Παραστάσεις - Χρονομετρήσεις

- Σειριακή εκτέλεση:

Total Execution time = 89,547 seconds

- 2 διεργασίες (4 νήματα ανά διεργασία):

	1 ^η εκτέλεση	2 ^η εκτέλεση	3 ^η εκτέλεση	4 ^η εκτέλεση	M.O
Communications time	0,01034	0,01033	0,01031	0,01037	0,0103
Computations time	55,83309	55,77853	55,58239	56,65983	55,96471
Total Execution time	55,84344	55,78886	55,59271	56,67021	55,97

Ποσοστό Communications Time	0,02%
Ποσοστό Computations time	99,98%

- 4 διεργασίες (4 νήματα ανά διεργασία):

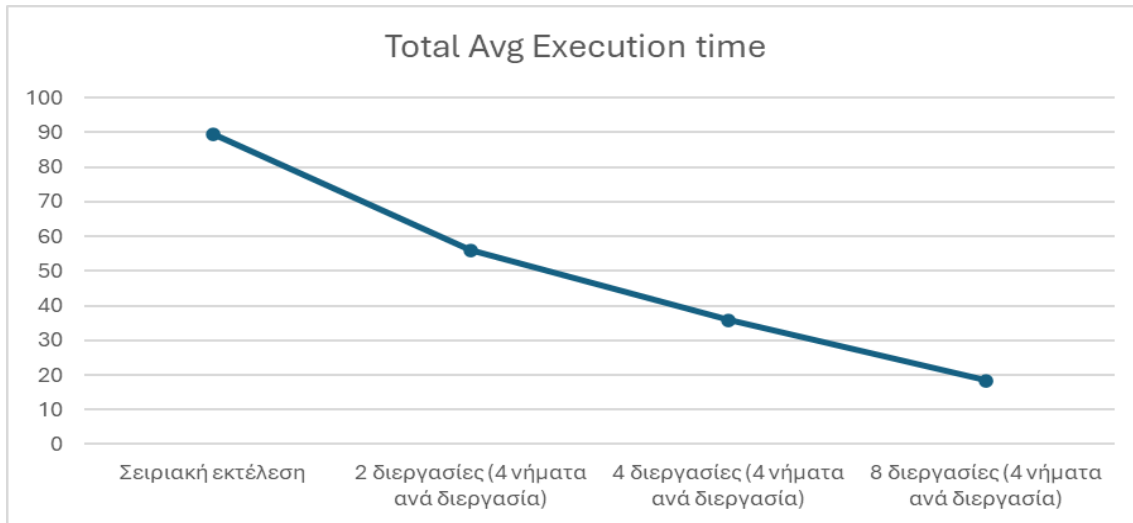
	1 ^η εκτέλεση	2 ^η εκτέλεση	3 ^η εκτέλεση	4 ^η εκτέλεση	M.O
Communications time	0,02242	0,02232	0,02264	0,02261	0,02249
Computations time	34,89871	35,63451	36,39073	36,02527	35,7373
Total Execution time	34,92114	35,65682	36,41337	36,04789	35,75

Ποσοστό Communications Time	0,06%
Ποσοστό Computations time	99,94%

- 8 διεργασίες (4 νήματα ανά διεργασία):

	1 ^η εκτέλεση	2 ^η εκτέλεση	3 ^η εκτέλεση	4 ^η εκτέλεση	M.O
Communications time	1,11414	1,43529	1,55444	0,87967	1,24588
Computations time	16,99832	17,41389	17,3027	17,08088	17,19894
Total Execution time	18,11246	18,84918	18,85714	17,96056	18,44

Ποσοστό Communications Time	6,75%
Ποσοστό Computations time	93,25%



Συμπεράσματα

1. Μείωση του Συνολικού Χρόνου Εκτέλεσης:

Η παράλληλη εκτέλεση μειώνει σημαντικά τον συνολικό χρόνο εκτέλεσης, καθώς αυξάνεται ο αριθμός των διεργασιών. Αυτό είναι αναμενόμενο, καθώς περισσότερες διεργασίες μοιράζονται το φορτίο υπολογισμών.

2. Αύξηση του Χρόνου Επικοινωνίας:

Παρατηρείται μια αύξηση στον χρόνο επικοινωνίας καθώς αυξάνεται ο αριθμός των διεργασιών. Αυτό οφείλεται στο γεγονός ότι περισσότερες διεργασίες απαιτούν περισσότερη επικοινωνία για τον συγχρονισμό και τη διανομή των δεδομένων.

Συνολικά, η παράλληλη εκτέλεση είναι εξαιρετικά αποδοτική για τον πολλαπλασιασμό πινάκων, με σημαντική μείωση του συνολικού χρόνου εκτέλεσης καθώς αυξάνεται ο αριθμός των διεργασιών.

Ωστόσο, η αύξηση του χρόνου επικοινωνίας υποδηλώνει ότι υπάρχει ένα όριο πέρα από το οποίο η περαιτέρω αύξηση των διεργασιών μπορεί να μην αποφέρει ανάλογα οφέλη λόγω του αυξημένου κόστους επικοινωνίας.

Άσκηση 3: Μονόπλευρες επικοινωνίες

Τι είναι οι Μονόπλευρες επικοινωνίες

Οι μονόπλευρες επικοινωνίες (one-sided communication) αποτελούν μια τεχνική που επιτρέπει τη μία διεργασία να έχει πρόσβαση απευθείας στη μνήμη μιας άλλης διεργασίας, χωρίς τη συμμετοχή της τελευταίας κατά τη στιγμή της πρόσβασης. Αυτό επιτυγχάνεται μέσω των λειτουργιών απομακρυσμένης προσπέλασης μνήμης (Remote Memory Access - RMA). Η βασική διαφορά από τις παραδοσιακές, αμφίπλευρες επικοινωνίες (two-sided communication) είναι ότι δεν απαιτείται η συμμετοχή της απομακρυσμένης διεργασίας για την αποστολή ή την λήψη δεδομένων.

Οι μονόπλευρες επικοινωνίες είναι ιδιαίτερα χρήσιμες σε περιβάλλοντα όπου η συγχρονισμένη αλληλεπίδραση μεταξύ διεργασιών είναι δύσκολη ή δεν είναι επιθυμητή, όπως σε εφαρμογές υψηλής απόδοσης που απαιτούν χαμηλή καθυστέρηση και μεγάλη ταχύτητα μεταφοράς δεδομένων.

Λειτουργίες του MPI για Μονόπλευρες επικοινωνίες

Οι βασικές λειτουργίες του MPI για μονόπλευρες επικοινωνίες περιλαμβάνουν:

1. **MPI_Put:**

Χρησιμοποιείται για να γράψει δεδομένα από τη μνήμη της τοπικής διεργασίας στη μνήμη μιας απομακρυσμένης διεργασίας.

2. **MPI_Get:**

Χρησιμοποιείται για να διαβάσει δεδομένα από τη μνήμη μιας απομακρυσμένης διεργασίας στη μνήμη της τοπικής διεργασίας.

3. **MPI_Accumulate:**

Χρησιμοποιείται για να εκτελέσει μια αναλυτική λειτουργία (όπως άθροιση) στα δεδομένα που βρίσκονται στη μνήμη μιας απομακρυσμένης διεργασίας.

Επιπλέον, το MPI παρέχει μηχανισμούς συγχρονισμού για να διασφαλίσει τη σωστή εκτέλεση των μονόπλευρων λειτουργιών:

1. MPI_Win_create:

Δημιουργεί ένα παράθυρο (window) στη μνήμη, το οποίο είναι προσπελάσιμο από άλλες διεργασίες μέσω RMA.

2. MPI_Win_fence:

Παρέχει ένα μηχανισμό συγχρονισμού για να διασφαλίσει ότι όλες οι προηγούμενες RMA λειτουργίες έχουν ολοκληρωθεί.

3. MPI_Win_lock/MPI_Win_unlock:

Επιτρέπουν τη χρήση αποκλειστικών και κοινών κλειδωμάτων για την πρόσβαση στη μνήμη άλλων διεργασιών.

Χρήση Μονόπλευρων Επικοινωνιών σε Εφαρμογές

Για να χρησιμοποιήσουμε μονόπλευρες επικοινωνίες σε μια εφαρμογή MPI, πρέπει να ακολουθήσουμε τα παρακάτω βήματα:

1. Αρχικοποίηση Παραθύρων Μνήμης:

```
MPI_Win win;
```

```
MPI_Win_create(baseptr, size, disp_unit, MPI_INFO_NULL, comm, &win);
```

2. Συγχρονισμός και Εκτέλεση RMA Λειτουργιών:

```
MPI_Win_fence(0, win); // Έναρξη RMA
```

```
MPI_Put(local_data, count, datatype, target_rank, target_disp, target_count,  
target_datatype, win);
```

```
MPI_Win_fence(0, win); // Τέλος RMA
```

3. Απελευθέρωση Παραθύρων Μνήμης:

```
MPI_Win_free(&win);
```

Πηγές

[One sided communication concepts](#)

[Roadmap: MPI One-Sided Communication](#)